

课程须知

➤ 本课程适用软件版本：MWORKS.Syslab2023b 1030

➤ 本课程示例运行需要软件加载：

相控阵工具箱

雷达工具箱

数学工具箱

基础工具箱

图形工具箱

MWORKS.Syslab相控阵、雷达工具箱应用

黄晨曦

苏州同元软控信息技术有限公司

2023-11-08



TONGYUAN
Software and Control

1 课程概述

创建、设计、测试和验证无线通信系统：

- 使用天线和天线阵列设计大规模 MIMO、毫米波和波束成形系统
- 使用室内和室外传播方案和信道模型在地图上评估现实世界的无线网络性能和指标
- 联合优化数字、射频和天线组件及模型，提高端到端系统性能

雷达工具箱

雷达系统工程

- 雷达方程
- 天线和接收机的增益和损耗
- 探测和跟踪统计
- 环境和杂波
- 合成孔径雷达

相控阵工具箱

相控阵设计和分析

- 天线、麦克风和声纳换能器
- 阵列几何结构与分析
- 信号发射和接收
- 发射机和接收机

检测、距离和多普勒分析

- 匹配滤波
- 拉伸处理的脉冲压缩
- 相干和非相干的脉冲累积
- 距离和多普勒估计
- 一维或二维的恒虚警率

波形设计和信号合成

- 脉冲波形
- 匹配滤波器和模糊度函数
- 运动建模和坐标系统
- 信号传播和目标

波束成形和波达方向估计

- 波束成形
- 波达方向估计
- 空时自适应处理

官网链接: [Syslab帮助文档 \(tongyuan.cc\)](https://tongyuan.cc)

目录

1.相控阵系统工程-阵列设计与分析

2.相控阵系统工程-波形赋型与DOA估计

3.相控阵系统工程-波形设计与信号合成

4.相控阵系统工程-检测/距离/多普勒估计

5.雷达系统工程

1.1 相控阵设计和分析 – 内容概述

相控阵作为天线、声纳的集合以**阵列模式**排列将信号转换为**电磁波**传输到目标，同时还可以将来自源或反射物体的输入能量转换为信号。阵列的性能在许多方面都超过了单个天线的性能，具体表现为一下三个主要优点：

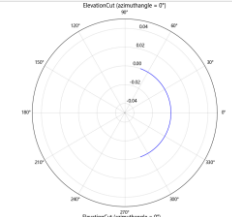
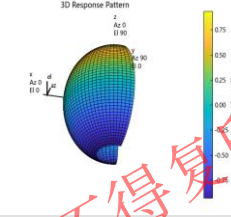
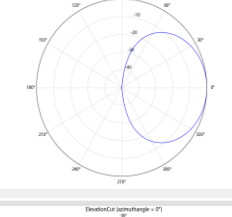
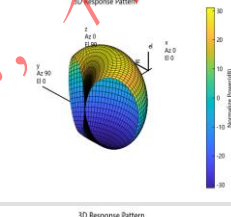
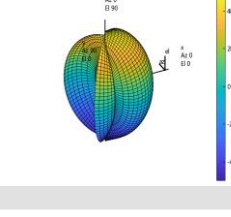
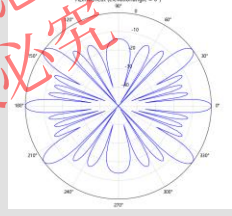
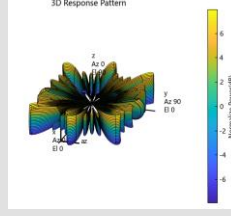
- **改进的空间分辨率** — 在定位和成像目标时查看更详细的分辨率；
- **电子转向** — 使用阵列转向矢量提高任何方向的检测性能；
- **干扰抑制** — 采用高级抑制技术，例如干扰消除、自适应波束成形和高分辨率测向。

阵列设计		阵列分析
天线/麦克风/声纳	阵列	phased_SteeringVector
phased_IsotropicAntennaElement	phased_UCA	phased_ArrayResponse
phased_CosineAntennaElement	phased_ULA	phased_ArrayGain
phased_CrossedDipoleAntennaElement	phased_URA	phased_ElementDelay
phased_CustomAntennaElement	phased_HeterogeneousConformalArray	isPolarizationCapable
phased_IsotropicHydrophone	phased_HeterogeneousULA	getElementPosition
phased_IsotropicProjector	phased_HeterogeneousURA	getNumElements
phased_NRAntennaElement		getTaper
phased_OmnidirectionalMicrophoneElement		collectPlaneWave
phased_ShortDipoleAntennaElement		



1.2 相控阵设计和分析 – 天线与阵列

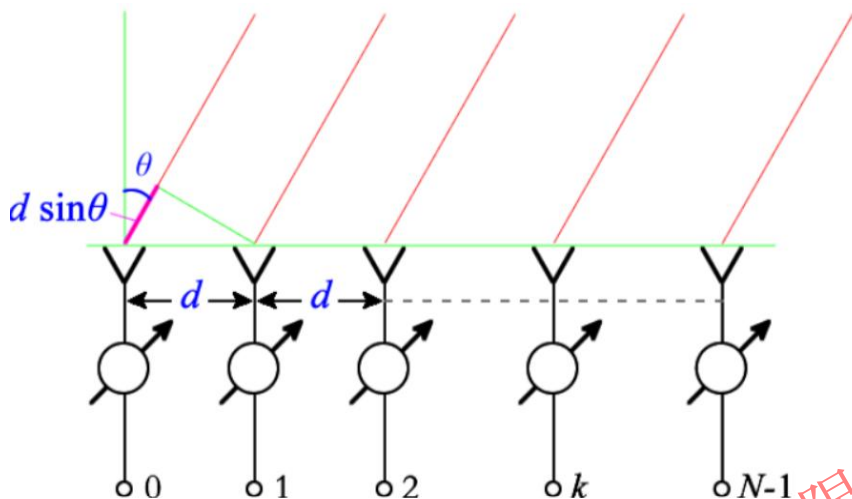
天线<典型天线与阵列辐射特性>

名称	二维辐射图	代码	三维辐射图	代码
各向同性天线		<pre>using TyPhasedArray antenna = phased_IsotropicAntennaElement(;FrequencyRange = [800e6,1.2e9]);fc = 1e9 pattern(antenna,fc,0, [-70:70];CoordinateSystem = "polar",Type = "powerdb",Normalize = true)</pre>		<pre>using TyPhasedArray antenna = phased_IsotropicAntennaElement(;FrequencyRange = [800e6,1.2e9]);fc = 1e9 pattern(antenna,fc,[-110:100], [-70:70];CoordinateSystem = "polar",Type = "powerdb",Normalize = true)</pre>
余弦天线		<pre>using TyPhasedArray antenna = phased_CosineAntennaElement(;FrequencyRange = [800e6 1.2e9],CosinePower = [1.5 2.5]) fc = 1e9 pattern(antenna,fc,0,[90:90];CoordinateSystem = "polar",Type = "powerdb")</pre>		<pre>using TyPhasedArray antenna = phased_CosineAntennaElement(;FrequencyRange = [800e6 1.2e9],CosinePower = [1.5 2.5]) fc = 1e9 pattern(antenna,fc,[-60:60],[-90:90];CoordinateSystem = "polar",Type = "powerdb")</pre>
交叉偶极子天线		<pre>using TyPhasedArray antenna = phased_CrossedDipoleAntennaElement(FrequencyRange = [1,2] * 1e9) fc = 1.5e9 pattern(antenna,fc,0,[-90:90];CoordinateSystem = "polar",Type = "powerdb",Polarization = "combined")</pre>		<pre>using TyPhasedArray antenna = phased_CrossedDipoleAntennaElement(FrequencyRange = [1,2] * 1e9) fc = 1.5e9 pattern(antenna,fc,[-130:130],[-90:90];CoordinateSystem = "polar",Type = "powerdb",Polarization = "H")</pre>
名称	二维辐射图	代码	三维辐射图	代码
均匀线阵		<pre>using TyPhasedArray array = phased_ULA(;NumElements = 5) fc = 1e9 c = physconst("LightSpeed") pattern(array,fc,[-180:180],0; PropagationSpeed = c, CoordinateSystem = "Polar", Type = "powerdb", Normalize = true)</pre>		<pre>using TyPhasedArray array = phased_ULA(;NumElements = 5) fc = 1e9 c = physconst("LightSpeed") pattern(array,fc,[-180:180],[-10:10]; PropagationSpeed = c, CoordinateSystem = "Polar", Type = "powerdb", Normalize = true)</pre>

1.3 相控阵设计和分析 – 阵列响应

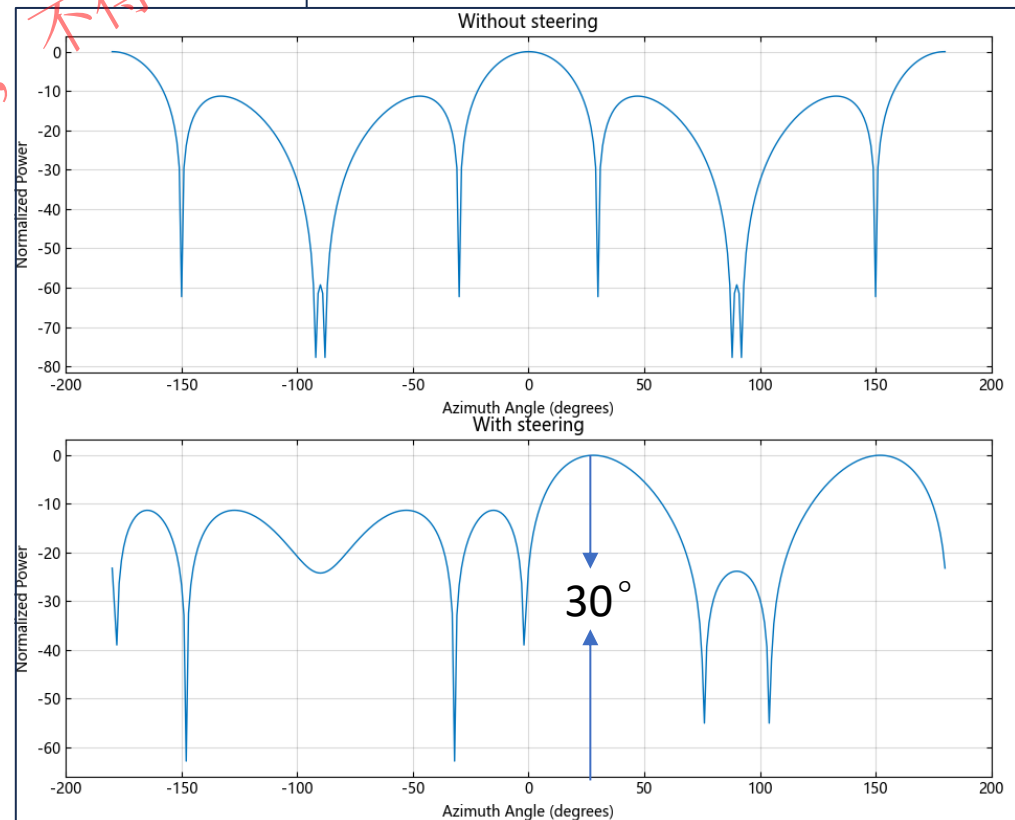
在毫米波信道中，通常采用天线阵列的**导向矢量**描述相邻接收天线之间信号的**相位差**。

阵列分析<导向矢量与阵列响应>

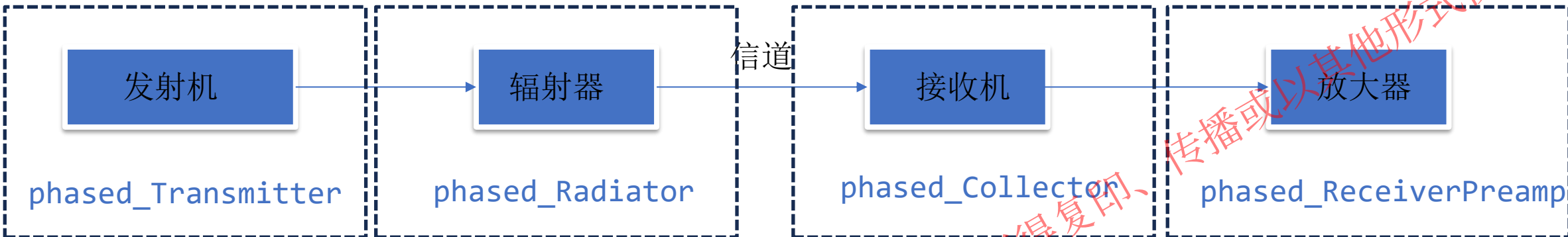


$$a = [1, e^{j\frac{2\pi}{\lambda}d\sin(\theta)}, \dots, e^{j\frac{2\pi}{\lambda}d(N-1)\sin(\theta)}]$$

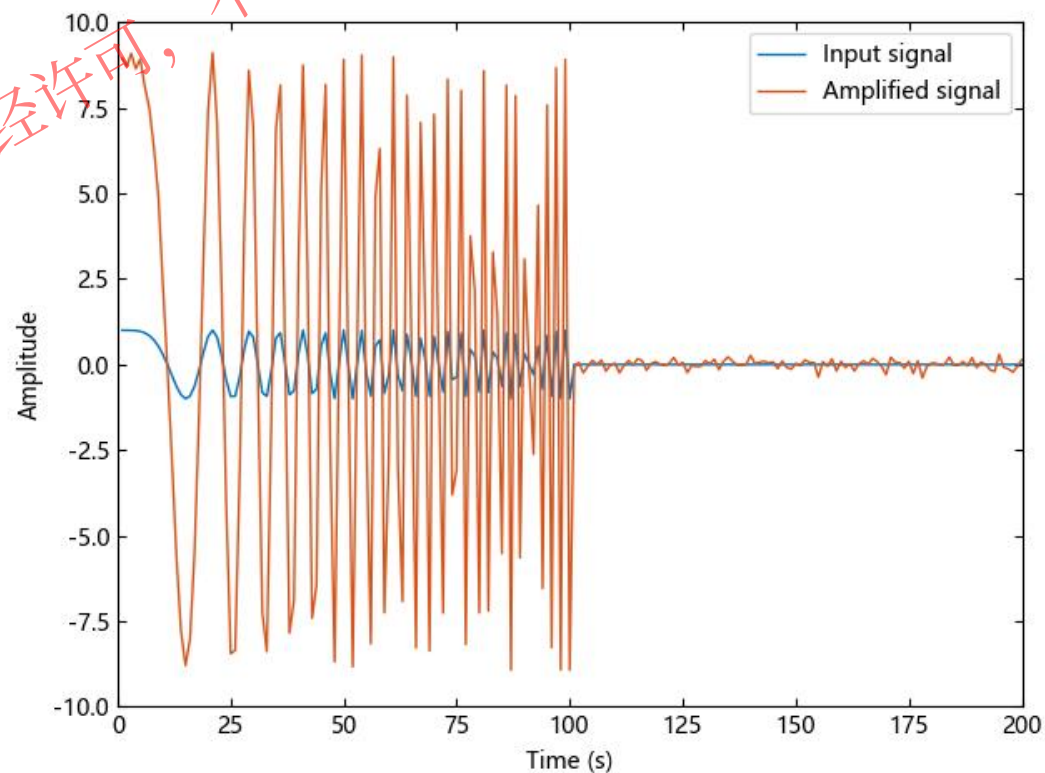
```
using TyPhasedArray
using TyPlot
fc = 300e6
c = physconst("LightSpeed")
array = phased_UA(; NumElements=4)
steervec1 = phased_SteeringVector(
SensorArray=array)
sv = step(steervec1, fc, [30; 20])
subplot(211)
pattern(
array,
fc,
[-180:180;],
0;
CoordinateSystem="rectangular",
PropagationSpeed=c,
Type="powerdb",
)
title("Without steering")
subplot(212)
pattern(
array,
fc,
[-180:180;],
0;
CoordinateSystem="rectangular",
PropagationSpeed=c,
Type="powerdb",
Weights=sv,
)
title("With steering")
```



1.4 相控阵设计与分析 – 信号发射与接收



```
using TyPhasedArray
using TyPlot
c = physconst("LightSpeed")
ann = phased_IsotropicAntennaElement()
fs = 1e7
waveform = phased_LinearFMWaveform(SampleRate = fs,
PulseWidth = 1e-5, SweepBandwidth = 5e6)
x, = step(waveform)
transmitter = phased_Transmitter(;PeakPower = 0.5)
sigtx = step(transmitter,x)
radiator = phased_Radiator(;Sensor = ann, PropagationSpeed =
c, OperatingFrequency = fs)
txsig = step(radiator,sigtx,[30;0])
collector = phased_Collector(;Sensor = ann, PropagationSpeed
= c, OperatingFrequency = fs)
noise = 0.1 * randn(1000,1) + im * randn(1000,1)
sigrx = step(collector,txsig + noise, [30;0])
receiver = phased_ReceiverPreamp(;SampleRate = fs, Gain = 2,
NoiseFigure = 1)
rxsig = step(receiver, sigrx)
plot(collect(1:1000),real(x),collect(1:1000),real(rxsig))
axis([0 200 -10 10])
xlabel("Time(s)")
ylabel("Amplitude")
legend("Input signal", "Amplified signal")
```



目录

1.相控阵系统工程-阵列设计与分析

2.相控阵系统工程-波形赋型与DOA估计

3.相控阵系统工程-波形设计与信号合成

4.相控阵系统工程-检测/距离/多普勒估计

5.雷达系统工程

2.1 波束成形和波达方向估计 – 波束赋形

波束赋形

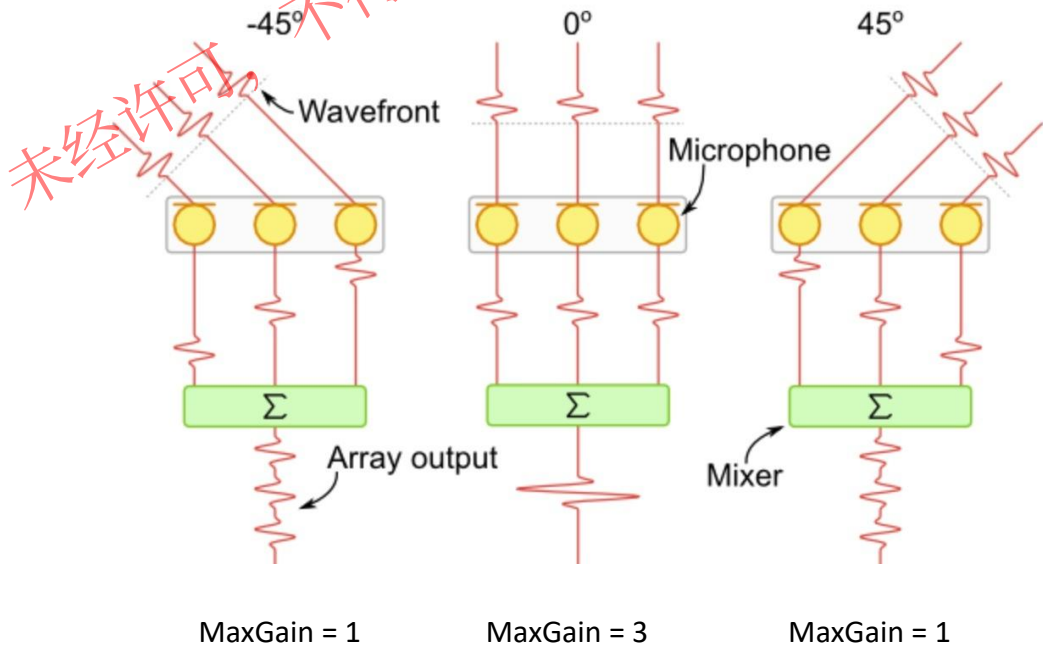
通过对多路信号进行合并，**抑制**非目标方向信号，**增强**目标方向信号，能够实现电磁波在特定方向的聚焦，进而能够有效提高接收信号的信噪比。

波束赋型<典型算法>

算法名称	
phased_LCMVBeamformer	窄带线性约束最小方差波束赋形器
phased_MVDRBeamformer	窄带最小方差无失真响应波束成形器
phased_PhaseShiftBeamformer	窄带相移波束赋形器
cbfweights	传统波束赋型权重
mvdrweights	最小方差无失真响应波束赋型权重
diagweights	对角化波束赋型权重

基本原理

通过调整不同阵列单元信号之间的参数，使得某些角度的信号**增强**，而另一些角度的信号相互**抵消**，例如下图中0度方向的输入信号同相位叠加后被增强。



2.1 波束成形和波达方向估计 – 波束赋形

➤ 基于线性约束最小方差 (LCMV) 准则的波束赋形

阵列输出: $y(t) = W^H x(t)$

方差: $E[|y(t)|^2] = W^H R_x W$

根据导向矢量 $a(\theta_0)$ 得到:

$x(t) = s(t)a(\theta_0) + j + n$

$y(t) = W^H x(t) = s(t)W^H a(\theta_0) + W^H(j + n)$

寻找最优的权重 W , 即固定信号分量 $W^H a(\theta_0) = 1$

最小化方差使干扰信号方差最小, 可得:

$$\begin{cases} \min W^H R_x W \\ s.t. W^H a(\theta_0) = 1 \end{cases}$$

可得 W 最优解:

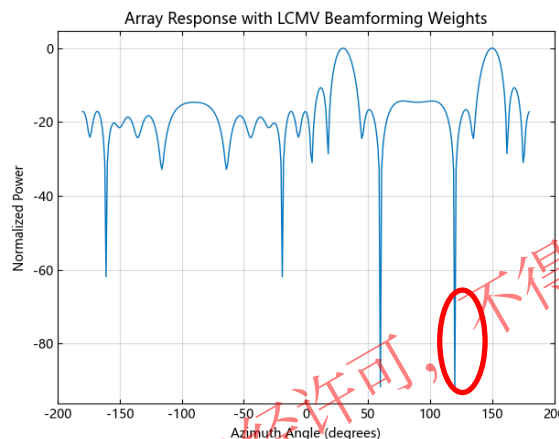
$$W = \mu R_x^{-1} a(\theta_0)$$

$$\mu = \frac{1}{a^H(\theta_0) R_x^{-1} a(\theta_0)}$$

推广到多个约束方向:

$$W = R_x^{-1} C (C^H R_x^{-1} C)^{-1} F$$

➤ LCMV波束赋型调零



阵列类型	ULA
天线类型	各向同性
阵元数目	10
阵元间距	$\lambda/2$
频率	1GHz
期望信号角度	[30;0]
干扰信号角度	[120;0]
快拍数	301

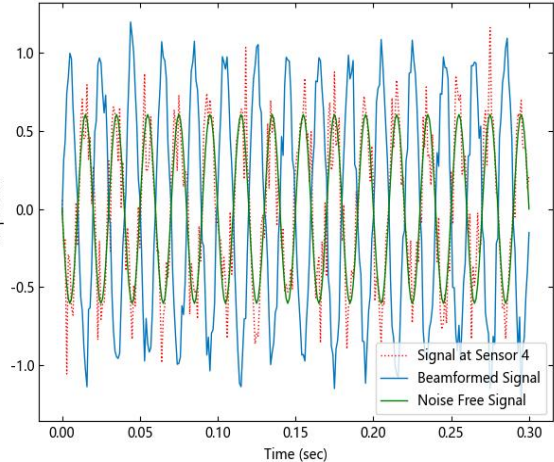
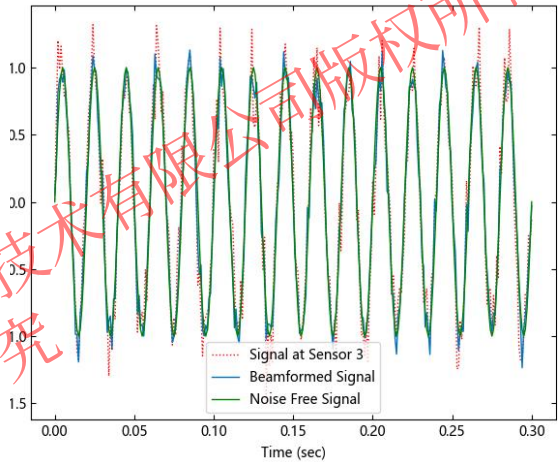
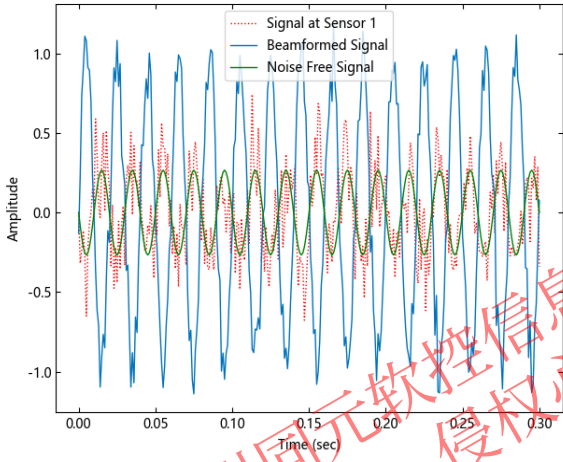
```
using TyPhasedArray
using TyPlot
fc = 1e9; lambda = physconst("LightSpeed") / fc
array = phased_ULA(NumElements = 10, ElementSpacing = lambda / 2)
array.Element.FrequencyRange = [8e8 1.2e9]
t = collect(range(0,0.3,300)); testsig = zeros(size(t)); testsig[201:205] = 1
angle_of_arrival = [30;0]
x = collectPlaneWave(array, testsig, angle_of_arrival, fc)
npower = 0.5
x = x + sqrt(npower / 2) * (randn(size(x)) + 1im * randn(size(x)))
jamsig = sqrt(10) * randn(300,10); jamsig_angle = [120;0]; noisePwr = 1e-5
noise = sqrt(noisePwr / 2) * (randn(size(jamsig)) + 1im * randn(size(jamsig)))
jamsig = jamsig + noise; rxsig = x + jamsig
steeringvector1 = phased_SteeringVector(SensorArray = array, PropagationSpeed =
physconst("LightSpeed"))
LCMVBeamformer = phased_LCMVBeamformer(DesiredResponse = 1, TrainingInputPort =
true, WeightsOutputPort = true)
LCMVBeamformer.Constraint = step(steeringvector1, fc, angle_of_arrival)
yLCMV, wLCMV = step(LCMVBeamformer, rxsig, jamsig)
pattern(array, fc, [-180:180], 0, PropagationSpeed = physconst("LightSpeed"), CoordinateSystem =
"rectangular", Type = "powerdb", Normalize = true, Weights = wLCMV)
title("Array Response with LCMV Beamforming Weights")
```

2.1 波束成形和波达方向估计 – 波束赋形

单个约束条件的LCMV波束赋形

将LCMV波束形成器应用于相控阵，并保留来自目标方向的信号。

阵列类型	ULA
天线类型	各向同性
阵元数目	5
频率	300MHz
期望信号角度	[45;0]
快拍数	301



```
using TyPhasedArray
using TyPlot
f = 50
t = collect(0:0.001:0.3)
x = sin.(2 * pi * f * t)
c = physconst("LightSpeed")
fc = 300e6
lambda = c / fc
incidentAngle = [45, 0]
antenna = phased_IsotropicAntennaElement(
    FrequencyRange=[20 20e8])
array = phased_ULA(; NumElements=5,
    ElementSpacing=lambda / 2, Element=antenna)
x = collectPlaneWave(array, x, incidentAngle, fc, c)
noise = 0.2*(randn(size(x)) + 1im * randn(size(x)))
rx = x + noise
steervecl = phased_SteeringVector(; SensorArray=array,
    PropagationSpeed=c)
beamformer = phased_LCMVBeamformer(
    steervecl, fc, incidentAngle,
    nse=1, WeightsOutputPort=true)
step(beamformer, rx)

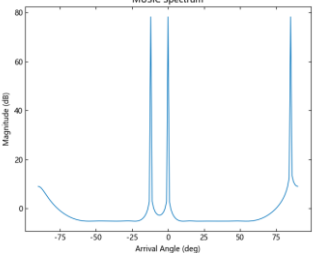
(rx[:,1]), "r:", t, real(y), t,
    "g")
(sec")
itude")
al at Sensor 1", "Beamformed Signal",
Signal")
```



2.2 波束成形和波达方向估计 – DoA估计

DoA估计的代表性方法是高分辨空间谱估计，其中包括阵列的信号模型、方向图、空时等效、MUSIC算法、MUSIC算法及改进、ESPRIT算法、LS-ESPRIT以及TLS-ESPRIT、空间平滑法等。

到达角估计<典型算法>

算法名称	原理介绍	示例代码	结果图示
Music(多重信号分类估计)	➢ 将任意阵列输出数据的协方差矩阵进行特征值分解 ➢ 对应不同特征值的特征向量构成相互正交的信号子空间和噪声子空间 ➢ 利用两个子空间之间的正交性来估计信号的来波方向 (DOA估计)	<pre>using TyPhasedArray;using TySignalProcessing; using TyPlot nelem = 11;d = 0.5;snr = 2.0 elementPos = collect([0:(nelem - 1)];') * d nsig = 3;angles = [0.0 -12.0 85.0] covmat = sensorcov(elementPos, angles, db2pow(-snr)) doas, spec, specang = musicdoa(covmat, nsig) plot(specang, 10 * log10.(spec));xlabel("Arrival Angle (deg)");ylabel("Magnitude (dB)");title("MUSIC Spectrum")</pre>	
Esprit(旋转不变子空间估计)	➢ 利用已知的数据估计自协方差函数 ➢ 由自协方差函数构造自协方差矩阵和互协方差矩阵 ➢ 求Rxx的特征值分解，以及噪声方差 ➢ 利用噪声方差计算矩阵Cxx和Cxy ➢ 求矩阵束{Cxx, Cxy}的广义特征值分解，进而估计波束到达角	<pre>using TyPhasedArray using TySignalProcessing N = 10 d = 0.5 elementPos = collect([(0:(N - 1));]') * d angles = [0 -25 30] Nsig = 3 R = sensorcov(elementPos, angles, db2pow(-5)) doa = espritdoa(R, Nsig)</pre>	<pre>3-element Vector{Float64}: 29.999999999999996 -25.0 5.615667336305956e-15</pre>
Rootmuisc(求根多重信号分类)	➢ 将任意阵列输出数据的协方差矩阵进行特征值分解 ➢ 对应不同特征值的特征向量构成相互正交的信号子空间和噪声子空间 ➢ 在MUSIC算法基础上，采用求根方法，求取N个接近于单位圆上的根估计信号的来波方向	<pre>using TyPhasedArray using TySignalProcessing N = 10 d = 0.5 elementPos = collect([0:(N - 1)];') * d angles = [0, -25, 30] Nsig = 3 R = sensorcov(elementPos, angles, db2pow(-5)) doa = rootmusicdoa(R, Nsig)</pre>	<pre>3-element Vector{Float64}: 29.999999874230642 -24.999999961021818 -5.858322432954794e-9</pre>

2.2 波束成形和波达方向估计 – DoA估计

基于ULA的DoA估计应用

DoA估计典型算法
phased_MUSICEstimator
phased_ESPRITEstimator
phased_BeamscanEstimator
phased_BeamspaceESPRITEstimator
phased_RootMUSICEstimator
musicdoa
espritdoa
aicest
mdltest
rootmusicdoa
spsmooth

	已知信源个数	未知信源个数
算法类型	phased_MUSICEstimator	phased_MUSICEstimator
阵列类型	均匀线阵	均匀线阵
阵元数目	10	10
频率	150MHz	150MHz
快拍数	8001	8001
信源估计算法	无	MDL/AIC
示例代码	<pre>using TyPhasedArray fc = 150.0e6; array = phased_ULA(, NumElements=10, ElementSpacing=1.0); fs=8000.0 t = collect((0:(1 / fs):1)) sig1 = cos.(2 * pi * t * 300.0) sig2 = cos.(2 * pi * t * 400.0) sig = collectPlaneWave(array, [sig1 sig2], [10 20; 60 -5]', fc); noise = 0.1 * (randn(size(sig)) + 1im * randn(size(sig))) estimator = phased_MUSICEstimator(SensorArray=array, OperatingFrequency=fc, DOAOutputPort=true, NumSignalsSource="Property", NumSignals=2,) _, doas = step(estimator, sig) az = broadside2az(sort(doas; dims=2), [20 -5])</pre>	<pre>using TyPhasedArray fc = 150.0e6; array = phased_ULA(, NumElements=10, ElementSpacing=1.0); fs=8000.0 t = collect((0:(1 / fs):1)) sig1 = cos.(2 * pi * t * 300.0) sig2 = cos.(2 * pi * t * 400.0) sig = collectPlaneWave(array, [sig1 sig2], [10 20; 60 -5]', fc); noise = 0.1 * (randn(size(sig)) + 1im * randn(size(sig))) estimator = phased_MUSICEstimator(SensorArray=array, OperatingFrequency=fc, DOAOutputPort=true, NumSignalsSource="Auto", NumSignalsMethod="MDL",) _, doas = step(estimator, sig) az = broadside2az(sort(doas; dims=2), [20 -5])</pre>
估计结果	[9.58288 60.3813]	[9.58288 60.3813]

目录

1.相控阵系统工程-阵列设计与分析

2.相控阵系统工程-波形赋型与DOA估计

3.相控阵系统工程-波形设计与信号合成

4.相控阵系统工程-检测/距离/多普勒估计

5.雷达系统工程

3.1 波形设计和信号合成 – 匹配滤波

使用 Hamming 窗口的匹配滤波器

怎么样的接收机频率响应 $H(\Omega)$ ，会得到最大的 SNR。将信号的瞬时功率

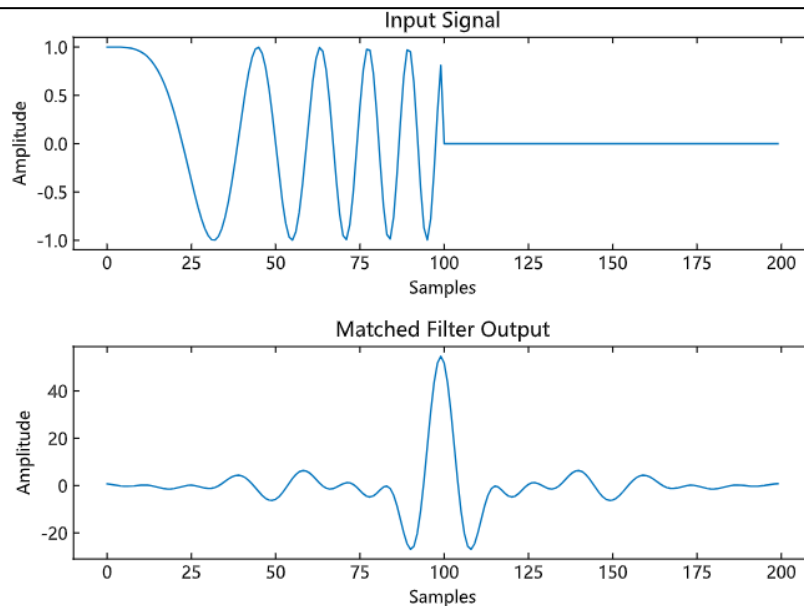
$$|y(T_M)|^2 = \left| \frac{1}{2\pi} \int_{-\infty}^{\infty} X(\Omega) H(\Omega) e^{i\Omega T_M} d\Omega \right|^2$$

通过施瓦兹不等式可以确定使其最大化的 $H(\Omega)$ 。当满足下式时，得到最大 SNR 值：

$$H(\Omega) = \alpha X^*(\Omega) e^{-j\Omega T_M} \quad \text{或} \quad h(t) = \alpha x^*(T_M - t)$$

这种选择接收机滤波器频率或者冲激响应的方式称为匹配滤波，因为响应与信号的波形时相匹配的。由此，我们可以通过频谱加权产生一个传递 $T(\Omega) = w(\Omega) H(\Omega)$ ，其中 $w(\Omega)$ 是窗函数频谱， $H(\Omega)$ 为匹配滤波器频谱。通过这个传递函数可以达到改善信噪比的目的。

```
using TyPhasedArray
using TyPlot
waveform = phased_LinearFMWaveform(; PulseWidth=1e-4, PRF=5e3)
x, ~, coeff = step(waveform)
filter = phased_MatchedFilter(; Coefficients=coeff,
SpectrumWindow="Hamming")
y, gain = step(filter, x)
subplot(2, 1, 1), plot(real(x))
xlabel("Samples")
ylabel("Amplitude")
title("Input Signal")
subplot(2, 1, 2), plot(real(y))
xlabel("Samples")
ylabel("Amplitude")
title("Matched Filter Output")
gcf().tight_layout()
```

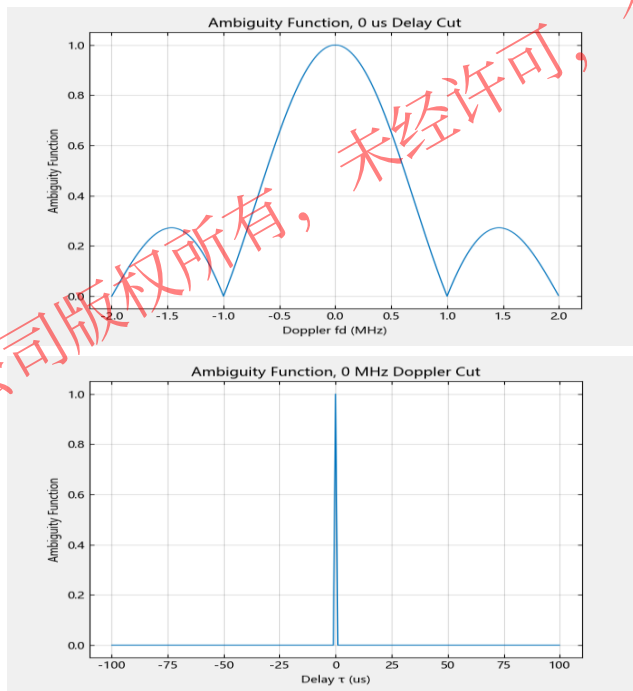
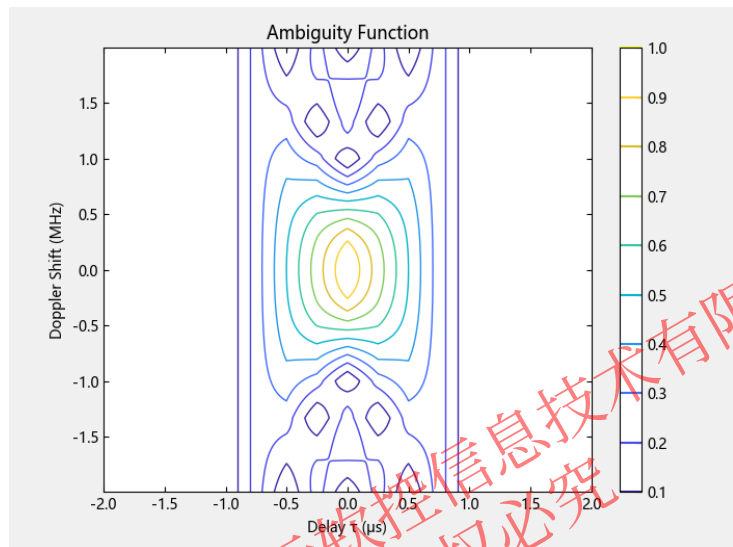


3.2 波形设计和信号合成 – 步进调频信号及其模糊函数

步进调频信号生成、求取其模糊函数并绘制

雷达发射的波形可以影响：目标检测、测量精度、分辨力、模糊度、杂波一致性等。模糊函数可以用来定性评价一个特定波形在实现这些能力方面到底有多好。多普勒相关函数表达式为：

$$\chi(\tau, \xi) = \int_{-\infty}^{\infty} u(t) \bar{u}(t + \tau) e^{j2\pi \xi t} dt$$



```
using TyPhasedArray
using TyMath
using TySignalProcessing
using TyPlot

Rmax = 15e3
Rres = 150
prf = 1 / range2time(Rmax)
bw = rangeres2bw(Rres)
tau = 1 / bw # Pulse width
fs = 4 * bw # Set the sampling frequency to a
             # high value to generate smoother plots
rectwav = phased_RectangularWaveform();
SampleRate=fs, PRF=prf, PulseWidth=tau)
rect_bw = bandwidth(rectwav)
rectsamples, = step(rectwav)
figure(1)
afmag, delay, doppler = ambgfun(rectsamples,
rectwav.SampleRate, rectwav.PRF)
ax = contour(vec(delay * 1e+6), vec(doppler *
1e-6), afmag)
xlabel("Delay τ (μs)")
ylabel("Doppler Shift (MHz)")
xlim([-tau tau] * 2e+6)
title("Ambiguity Function")
colorbar(ax)
ambgfun(rectsamples, fs, prf, Cut="Doppler";
plotfig=true)
ambgfun(rectsamples, fs, prf, Cut="Delay";
plotfig=true)
```

目录

1.相控阵系统工程-阵列设计与分析

2.相控阵系统工程-波形赋型与DOA估计

3.相控阵系统工程-波形设计与信号合成

4.相控阵系统工程-检测/距离/多普勒估计

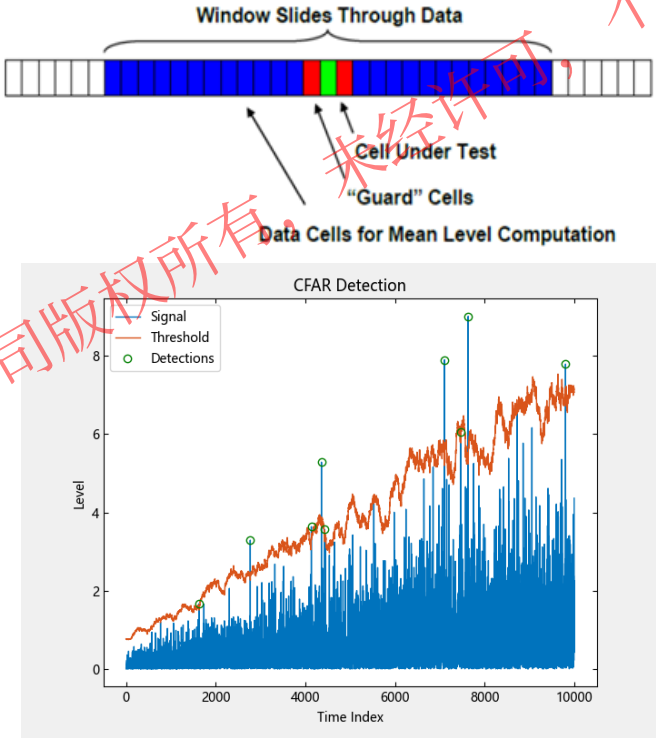
5.雷达系统工程

4.1 检测、距离和多普勒分析 – 恒虚警检测

基于CA方法的恒虚警检测

雷达信号的检测总是在**干扰背景**下进行，这些干扰包括接收机内部热噪声以及来自地面、雨雪、海浪等杂波干扰，甚至是人为释放的有源和无源干扰。恒虚警处理的目的是在上述恶劣的干扰下保持信号检测时的**虚警率恒定**，以避免过多的虚假数据进入信息处理系统而影响其工作效率，同时保证有用数据尽可能没有丢失地进入信息处理系统。

方法	噪声估算	适用场景	缺点
"CA"	使用所有训练单元中值的平均值	均匀杂波背景	杂波边缘处虚警率上升，多目标环境下检测性能下降
"GOCA"	选择前部训练单元和后部训练单元中的较大平均值	在杂波边缘和均匀杂波环境能保持较好检测性能	多目标环境下检测性能下降
"SOCA"	选择前部训练单元和后部训练单元的平均值中较小的一个	在干扰目标位于前沿或者后沿滑窗之一的多目标环境中能分辨出主目标	杂波边缘处和均匀杂波环境中检测性能差
"OS"	按升序对训练单元中的值进行排序。选择第 N 个单元，其中 N 是 Rank 属性的值	多目标检测性能较好	杂波边缘区域虚警概率提高；且依赖于参考窗内所有样本数据，Rank 取值直接决定了结果优劣



```
using TyPhasedArray
using TyMath
using TySignalProcessing
using TyPlot

cfar = phased_CFARDetector()
exp_pfa = 1e-3;
cfar.ThresholdFactor = "Auto"
cfar.NumTrainingCells = 200
cfar.ProbabilityFalseAlarm = exp_pfa

rs = MT19937ar(2010)
npower = db2pow(-10) # Assume 10dB SNR ratio

Npoints = 10000
CUTIdx = 1

# Noise samples after a square law detector
rsamp = randn(rs, Npoints, 1) + 1im * randn(rs, Npoints, 1)
ramp = collect(range(1, stop=10, length=Npoints))
xRamp = abs2.(sqrt.(npower .* ramp ./ 2) .* rsamp)

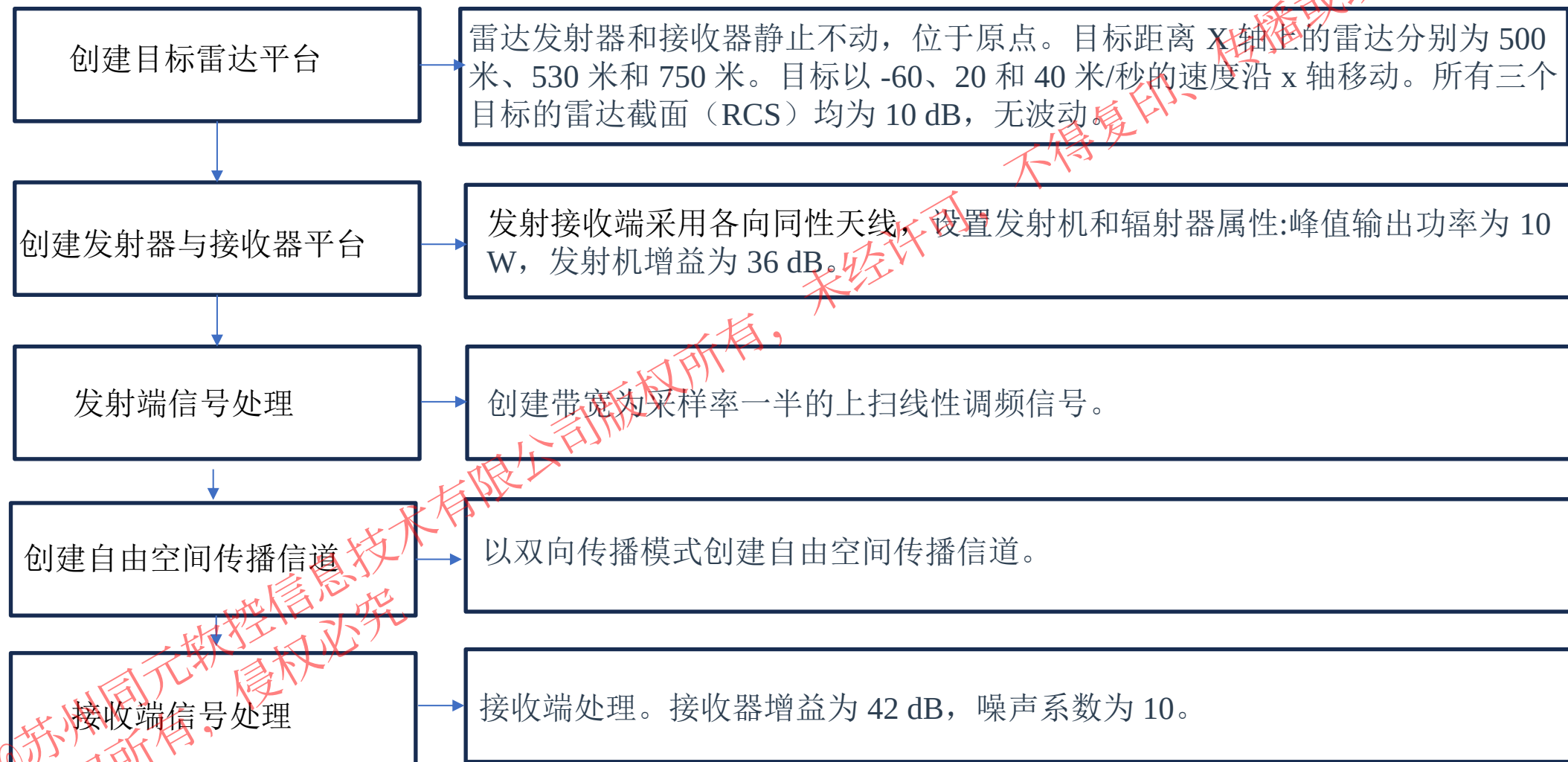
x_detected, th, noise = step(cfar, xRamp, 1:length(xRamp))

indx = collect(i[1] for i in findall(x_detected))
plot(1:length(xRamp), xRamp, 1:length(xRamp), th, indx,
xRamp[x_detected], "go")
legend("Signal", "Threshold", "Detections", "Location",
"Northwest")
xlabel("Time Index")
ylabel("Level")
title("CFAR Detection")
```



4.2 检测、距离和多普勒分析 – 范围响应

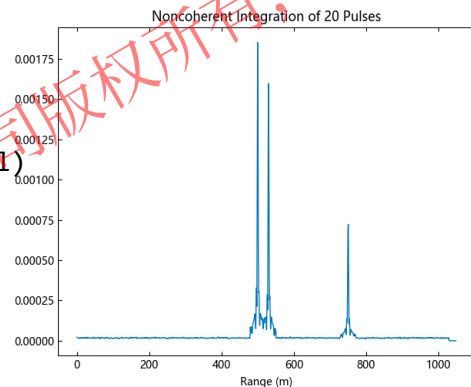
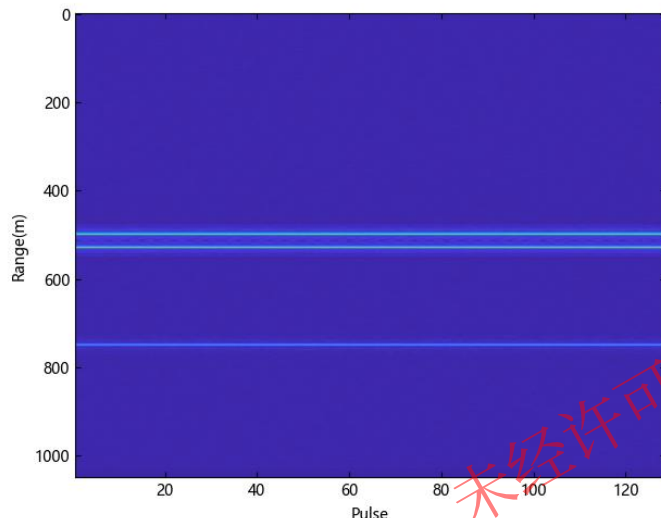
三个目标雷达范围响应



4.2 检测、距离和多普勒分析 – 范围响应

目标雷达范围响应

```
using TyPlot
using TyPhasedArray
using TySignalProcessing
using TyCommunication
fs = 150e6
c = physconst("LightSpeed")
fc = 77.0e9
pri = 7e-6
prf = 1 / pri
Numtgts = 3
tgtpos = zeros(Numtgts, Numtgts)
tgtpos[1:1, :] = [500 530 750]
tgtvel = zeros(3, Numtgts)
tgtvel[1:1, :] = [-60 20 40]
tgtcrs = db2pow(10) * [1 1 1]
tgtmotion = phased_Platform(; pos=tgtpos, vel=tgtvel)
target = phased_RadarTarget(; PropagationSpeed=c,
OperatingFrequency=fc, MeanRCS=tgtcrs)
radarpos = [0; 0; 0]
radarvel = [0; 0; 0]
radarmotion = phased_Platform(; pos=radarpos, vel=radarvel)
txantenna = phased_IsotropicAntennaElement()
rxantenna = txantenna
bw = fs / 2
waveform = phased_LinearFMWaveform(;
    SampleRate=fs,
    PRF=prf,
    OutputFormat="Pulses",
    NumPulses=1,
    SweepBandwidth=fs / 2,
    DurationSpecification="Duty cycle",
    DutyCycle=0.02,
)
```



```
sig, = step(waveform)
Nr = length(sig)
bwrms = bandwidth(waveform) / sqrt(12)
rngrms = c / bwrms
peakpower = 10
txgain = 36.0
txgain = 36.0
transmitter = phased_Transmitter(; PeakPower=peakpower,
Gain=txgain, InUseOutputPort=true)
radiator = phased_Radiator(; Sensor=txantenna,
PropagationSpeed=c, OperatingFrequency=fc)
channel = phased_FreeSpace(; SampleRate=fs,
PropagationSpeed=c, OperatingFrequency=fc,
TwoWayPropagation=true)
collector = phased_Collector(; Sensor=rxantenna,
PropagationSpeed=c, OperatingFrequency=fc)
rxgain = 42.0
noisefig = 1
receiver = phased_ReceiverPreamp(; SampleRate=fs,
Gain=rxgain, NoiseFigure=noisefig)
figure(1)
imagesc(
    abs.(cube);
    xvalue=collect(0:(Np - 1)) * pri * 1e6,
    yvalue=collect(0:(Nr - 1)) / fs * 1e6,
)
xlabel("Slow Time/μs")
ylabel("Fast Time/μs")
axis("xy")
figure(2)
matchingcoeff = getMatchedFilter(waveform)
ndop = 128
rangeresp = phased_RangeResponse(; SampleRate=fs,
PropagationSpeed=c)
resp, rnggrid = step(rangeresp, cube, matchingcoeff)
imagesc(abs.(resp); xvalue=collect(1:Np),
yvalue=collect(rnggrid))
xlabel("Pulse")
ylabel("Range (m)")
```

目录

1.相控阵系统工程-阵列设计与分析

2.相控阵系统工程-波形赋型与DOA估计

3.相控阵系统工程-波形设计与信号合成

4.相控阵系统工程-检测/距离/多普勒估计

5.雷达系统工程

1.1 雷达方程-内容概述

基本雷达， SAR雷达， 搜索雷达方程

雷达方程相关函数

radareqpow

radareqrng

radareqsnr

radareqsarsnr

radareqsarrng

radareqsarpow

radareqsearchpap

radareqsearchrng

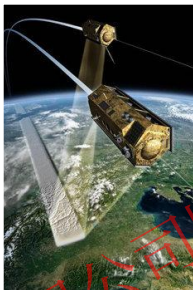
radareqsearchsnr

radarmetricplot

.....



雷达基本方程，也叫做最大作用距离方程，研究雷达方程主要有以下作用：1.根据雷达参数来估算雷达的作用距离；2.根据雷达的威力范围来估算雷达的发射功率；3.分析雷达参数对雷达作用距离的影响，这对雷达系统设计中正确地选择系统参数有重要的指导作用



SAR雷达 (Synthetic Aperture Radar)，是一种主动式的对地观测系统，可安装在飞机、卫星、宇宙飞船等飞行平台上，全天时、全天候对地实施观测、并具有一定的地表穿透能力。利用合成孔径原理，实现高分辨的微波成像，具备全天时、全天候、高分辨、大幅宽等多种特点。



搜索雷达的主要任务是发现远距离的军事目标，需要很远的探测距离和很大的覆盖空域。由于要求探测距离远，搜索雷达一般采用较低的频段，通常是L和S频段，有时是UHF，特殊情况下是VHF，甚至HF频段。在这些频段上，大气衰减不太大，杂波反射率低，例如承担预警任务的雷达

1.2 雷达方程-应用示例

雷达作用距离

```
using TyPhasedArray
using TyRadar
lambda =
physconst("LightSpeed")/10e9; #
波长
SNR = 6; # 信噪比
tau = 10e-6; # 脉宽
Pt = 1e6; # 发射功率
RCS = 0.1; # 雷达目标RCS
Gain = 40; # 收发天线增益
Loss = 3; # 综合损耗
maxrng2 =
radareqrng(lambda,SNR,Pt,tau,Gai
n=Gain, RCS=RCS, Loss=Loss) #最大
可探测距离
```

所需发射功率

```
using TyPhasedArray
using TyRadar
fc = 10.0e9; # 载波频率
lambda =
physconst("LightSpeed")/fc;
SNR = 6; # 信噪比
tau = 10e-6; # 脉宽
TxRng = 50e3; # 发射机到目标距离
RvRng = 75e3; # 接收机到目标距离
TxRvRng = [TxRng RvRng];
TxGain = 40; # 发射天线增益
RvGain = 20; # 接收天线增益
Gain = [TxGain RvGain];
Pt =
radareqpow(lambda,TxRvRng,SNR,tau
;Gain=Gain) #所需发射功率
```

接收机信噪比

```
using TyPhasedArray
using TyRadar
fc = 10.0e9; # 载波频率
lambda =
physconst("LightSpeed")/fc; #
波长
tau = 1e-6; # 脉宽
Pt = 1e6; # 发射功率
txrvRng = [50e3 75e3]; # 接收机
发射机到目标距离
Gain = [40 20]; # 收发天线增益
snr =
radareqsnr(lambda,txrvRng,Pt,tau
,Gain=Gain) # 接收机最小输出信噪比
```

2.1 目标检测—Blake图，雷达性能指标图

检测和跟踪相关函数

detectability

probgrid

radarmetricplot

radarvcd

rocinterp

rocpfa

rocsnr

shnidman

toccgh

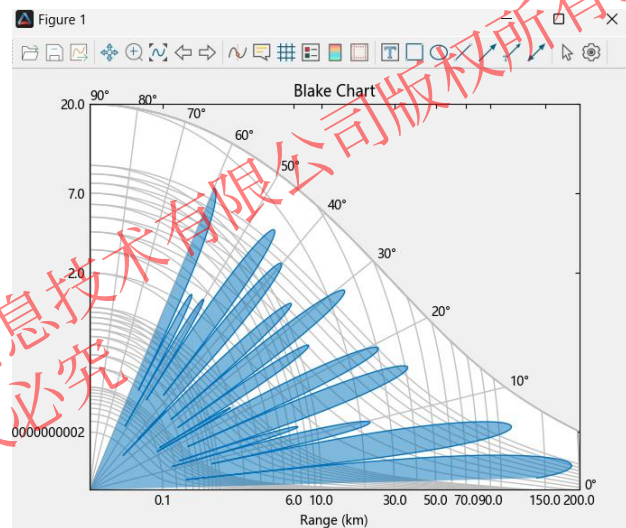
albersheim

.....

Blake图显示了不同目标海拔角情况下的最大雷达范围。同时显示了常数范围线和常数高度线。

可以调用blake函数来绘制，输入为radarvcd函数生成的垂直覆盖模式vcp和垂直覆盖模式角度vcpangles。

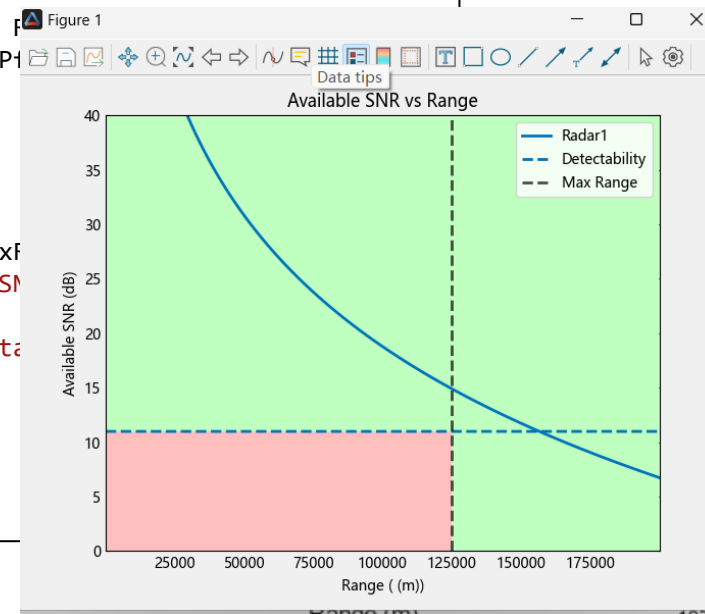
```
using TyRadar
freq = 100e6;
ant_height = 20;
rng_fs = 100;
vcp, vcpangles =
radarvcd(freq, rng_fs, ant_height);
blakechart(vcp, vcpangles);
```



雷达性能指标图，以目标距离为横坐标。信噪比为纵坐标。标记所需的最大检测范围和检测因子，使用红绿颜色代表系统在不同范围内的检测性能。绿色代表满足要求，红色代表不满足要求。

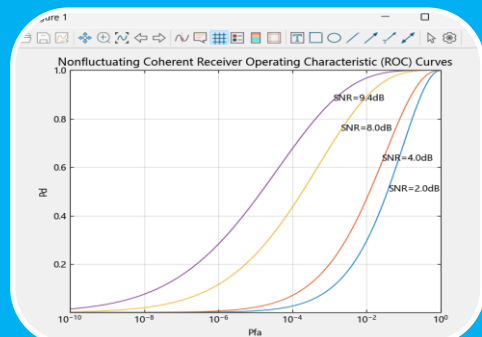
```
using TyRadar
using TyPhasedArray
using TyPlot
lambda, = freq2wavelen(3e9)
Pt = 5e3
tau = 1.2e-5
N = 24
SwerlingCase = "Swerling1"
G = 40;
Pfa = 1e-6;
Pd = 0.9;
MaxRangeRq = 125e3;
R = (1:1e2:200e3)
SNRav = radareqsnr(lambda, Pt, tau, N, G, Pfa, Pd)
DxObj = detectability(Pd, Pfa, SNRav)
radarmetricplot(
    R,
    SNRav,
    DxObj;
    MaxRangeRequirement=MaxRangeRq;
    MetricName="Available SNR";
    MetricUnit="dB";
    RequirementName="Detectability";
    ShowStoplight=true,
)
ylim([0 40])
```

波长 (m)
功率 (W)
脉宽 (s)
接受脉冲个数
Swerling模型
天线增益 (dB)
虚警概率
目标检测概率
最大探测距离 (m)



2.2检测和跟踪—ROC曲线

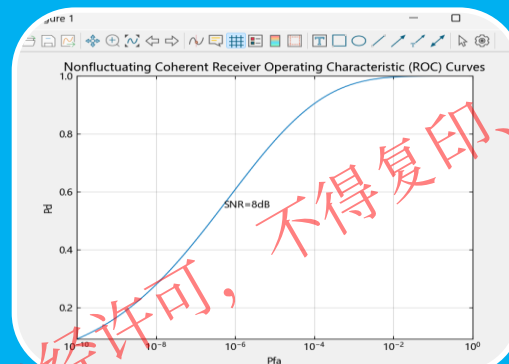
接收机工作特征曲线(ROC)通常用于评估雷达或声纳探测器的性能。是在给定信噪比的情况下，检测概率与误警概率的曲线图。



```
SNRvals = [2 4 8 9.4];  
rocsnr(SNRvals;plotfig=true);
```

- 指定一个SNR值的向量，rocsnr函数会为每个SNR值计算ROC曲线，对给定的SNR值分别计算Pd和Pfa值

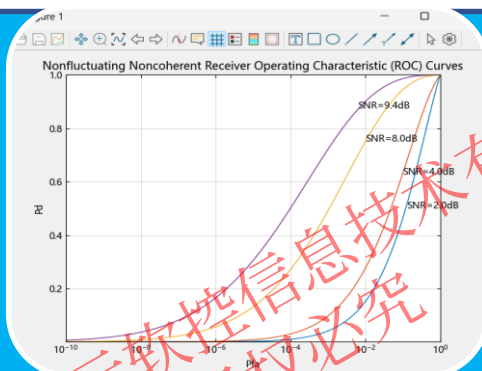
- 将plotfig参数置为true，能够直接画出对比曲线



```
rocsnr([8],NumPulses=2;plotfig=true);
```

- 提高探测性能的一种方法是对几个脉冲进行平均。这在信号已知且出现在加性复白噪声中的情况下特别有用

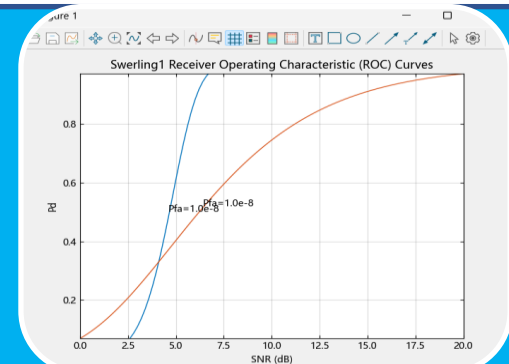
- 通过改变rocsnr的输入参数来构建此场景；假设信噪比为8 dB并对两个脉冲进行平均来分析性能。



```
SNRvals = [2 4 8 9.4];  
rocsnr(SNRvals,SignalType="NonfluctuatingNoncoherent",plotfig=true);
```

- rocsnr函数默认假设为相干检测。为了分析在信号已知但相位未知的情况下的性能，您可以更换SignalType参数

- 使用与之前相同的信噪比值，分析非相干检测的性能



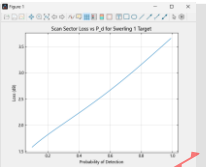
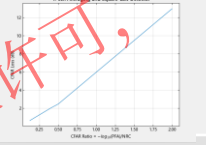
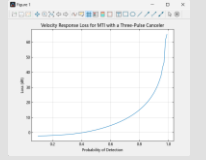
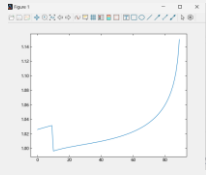
```
rocpfa([1e8],NumPulses=10,SignalType="NonfluctuatingNoncoherent",plotfig=true);hold("on")  
rocpfa([1e8],NumPulses=10,SignalType="Swierling1",plotfig=true);
```

- 想要比较一下非起伏目标和Swierling 1目标的ROC曲线，并探究在要达到相同的Pd和Pfa时的信噪比。

- 可使用rocpfa函数绘制为在不同Pfa下的ROC曲线，SNR为横轴，Pd为纵轴。

3.1增益和损耗

常用损耗函数

名称ni	功能	图例	代码
arrayscanloss	天线扫描损耗：在天线进行扫描工作时，由于辐射能量的方向性和扫描速度的影响，导致天线辐射功率的损耗。		<pre>Pd = 0.1:0.01:0.99;Pfa = 1e-6;N = 1; ThetaM = [-45 70]; Lss = arrayscanloss(Pd, Pfa, N, ThetaM, "Swerling1"); plot(Pd, Lss)</pre>
cfarloss	CFAR检测损耗：是由检测门限非理想估计值与理想的门限相比所造成的。估计的波动迫使门限均值高于理想门限值，因而产生了损耗。		<pre>PFA = 1e-8 NRC = 4:4:64 LCFAR = cfarloss(PFA, NRC) plot(-log10(PFA) ./ NRC, LCFAR)</pre>
mtiloss	MTI损耗：由于目标的速度会导致回波信号的频率发生变化，从而带来损耗。这会导致雷达系统在检测和跟踪运动目标时的性能下降，因为一部分回波信号会由于频率变化而被滤波器剔除掉		<pre>PFA = 1e-6;N = 24;M = 3; PD = 0.1:0.01:0.99; _, LV = mtiloss(PD, PFA, N, M); plot(PD, LV)</pre>
binaryintloss	脉冲积累损耗：脉冲积累是指将多个连续的脉冲信号进行叠加和累积，以增加雷达系统的探测灵敏度和距离分辨能力，但由于脉冲信号的累积作用而导致接收到的回波信号强度减弱，进而导致了损耗。		<pre>PD = 0.1:0.01:0.99;PFA = 1e-6; N = 24;M = 12; out, = binaryintloss(PD, PFA, N, M) plot(out)</pre>

3.2增益和损耗——脉冲积累的增益损耗

相干积累与非相干积累

#相干积累和费相干积累的增益

```
N = (1:100)'; # Number of received pulses
Pd = [0.5; 0.95]; # Detection probability
Pfa = [1e-4 1e-8]; # Probability of false alarm
D0 = detectability(Pd,Pfa,1); # Single-pulse detectability factor
numPdPfa = length(Pd)*length(Pfa);

D0n = zeros(numPdPfa,length(N));# N-pulse detectability
for i = 1:length(N)
    D0n[:,i] = vec(detectability(Pd,Pfa,N[i]))
end

Gnc = reshape(D0, :, 1) ./ D0n; # Noncoherent integration gain
Gc = pow2db(N); # Coherent integration gain
figure()
semilogx(N,Gnc[1:2,:]',linewidth=2)
hold("on")
semilogx(N,Gnc[3:4,:]', "--",linewidth=2)
semilogx(N,Gc,"k",linewidth=2)
semilogx(N,pow2db(sqrt.(N)), "k:",linewidth=2)
xlabel("Number of Integrated Pulses")
ylabel("Gain (dB)")
title("Pulse Integration Gain")
legend("Noncoherent Pd=$(Pd[1]),Pfa=$(Pfa[1])", "Noncoherent Pd=$(Pd[1]),Pfa=$(Pfa[2])", "Noncoherent Pd=$(Pd[2]),Pfa=$(Pfa[1])", "Noncoherent Pd=$(Pd[2]),Pfa=$(Pfa[2])", "Coherent",raw"$N^{\{1/2\}}$")
grid("on")
```

二进制积累

二进制积累，是一种双阈值系统。第一个阈值应用于每个脉冲，得到一个检测结果，每个结果都有检测概率 p_d 和虚警概率 p_{fa} 。将这些结果组合并与第二个阈值进行比较。至少M个脉冲越过第一个阈值，则会判定存在目标

检测概率和虚警概率得到的二进制积累损耗如下

```
#二进制积累
# Binary integration loss
Lb = zeros(numPdPfa,length(N));

for i = 1:length(N)
    Lb[:,i] = reshape(binaryintloss(Pd,Pfa,N[i])[1],:,1);
end

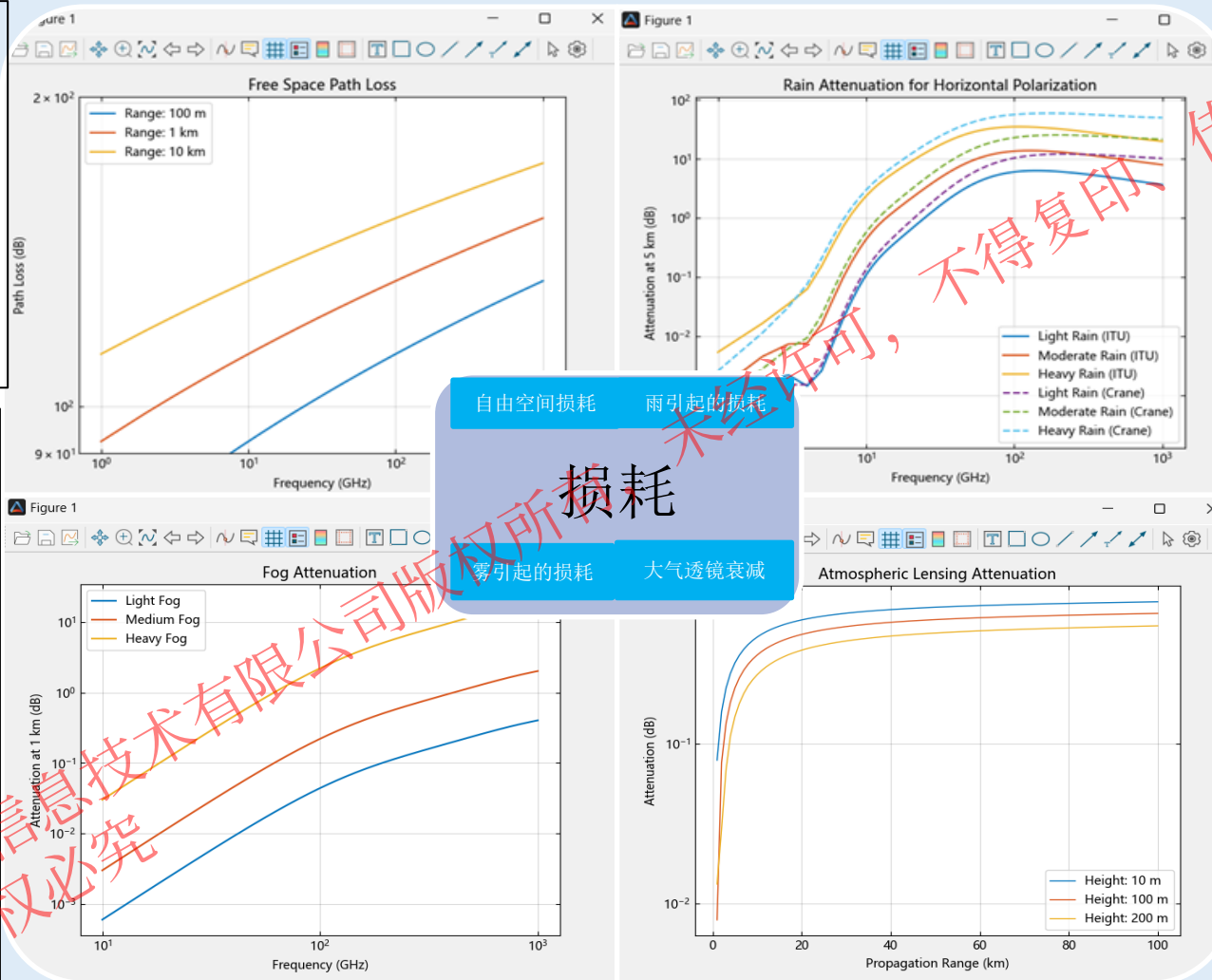
figure()
semilogx(N,Lb',linewidth=2)

xlabel("Number of Integrated Pulses")
ylabel("Loss (dB)")
title("Binary Integration Loss"*raw"$M=0.955N^{\{0.8\}}$")
legend("Noncoherent Pd=$(Pd[1]),Pfa=$(Pfa[1])", "Noncoherent Pd=$(Pd[1]),Pfa=$(Pfa[2])", "Noncoherent Pd=$(Pd[2]),Pfa=$(Pfa[1])", "Noncoherent Pd=$(Pd[2]),Pfa=$(Pfa[2])", "Coherent",raw"$N^{\{1/2\}}$")
grid("on")
```

4.1 环境和杂波——不同环境引起的损耗

```
c = physconst("lightspeed");
R0 = [100; 1e3; 10e3];
freq = (1:1000)*1e9;
apathloss = fspl(R0,c./freq);
loglog(freq/1e9,apathloss);
grid("on");
ylim([90 200]);
legend("Range: 100 m", "Range: 1 km", "Range: 10 km");
xlabel("Frequency (GHz)");
ylabel("Path Loss (dB)");
title("Free Space Path Loss");
```

```
freq = (10:1000)*1e9;
T = 15;
waterdensity = [0.01 0.05 0.5];
fogloss =
zeros(length(freq),length(waterdensity))
for m = 1:length(waterdensity)
    fogloss[:,m] =
fogpl(R0,freq,T,waterdensity[m])
';
end
loglog(freq/1e9,fogloss);
grid("on");
legend("Light Fog", "Medium Fog", "Heavy Fog");
xlabel("Frequency (GHz)");
ylabel("Attenuation at 1 km (dB)");
title("Fog Attenuation");
```



```
R0 = 5e3; # 5 km range
rainrate = [1 20]; # rain rate in mm/h
el = 0; # 0 degree elevation
tau = 0; # horizontal polarization
freq = (1:1000)*1e9;
rainloss_itu = zeros(1000,length(rainrate))
rainloss_crane = zeros(1000,length(rainrate))
for m = 1:length(rainrate)
    rainloss_itu[:,m] = rainpl(R0,freq,rainrate[m],el,[tau]);
    rainloss_crane[:,m] = crainerainpl(R0,freq,rainrate[m],el,[tau]);
end
loglog(freq/1e9,rainloss_itu);
hold("on");
loglog(freq/1e9,rainloss_crane,"--");
hold("off");
grid("on");
legend("Light Rain (ITU)", "Moderate Rain (ITU)", "Heavy Rain (ITU)",
"Light Rain (Crane)", "Moderate Rain (Crane)", "Heavy Rain (Crane)");
xlabel("Frequency (GHz)");
ylabel("Attenuation at 5 km (dB)");
title("Rain Attenuation for Horizontal Polarization");
```

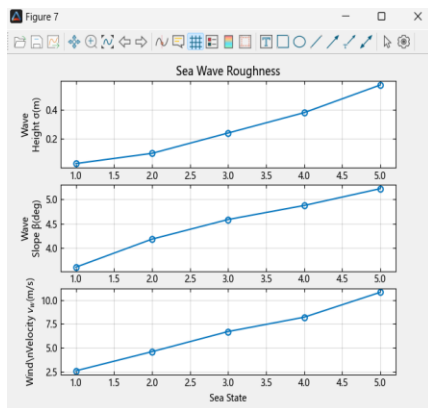
```
R = 1e3:1e3:100e3; # propagation range
range
el = 0.05; # elevation angle
heights = [10 100 200]; # radar platform heights
lenloss =
zeros(length(R),length(heights),)
for m = 1:length(heights)
    lenloss[:,m] =
lenspl(R,heights[m],el);
end
semilogy(R/1e3,lenloss);
grid("on");
legend("Height: 10 m", "Height: 100 m", "Height: 200 m");
xlabel("Propagation Range (km)");
ylabel("Attenuation (dB)");
title("Atmospheric Lensing Attenuation");
```

4.2环境和杂波——海杂波仿真

在对海杂波进行建模，需要去描述海平面状态，可通过下面三个参数描述

- 1.海浪的高度 σ
- 2.海浪的坡度 β
- 3.风速 v_w

可使用searoughness函数得到不同海况下的参数

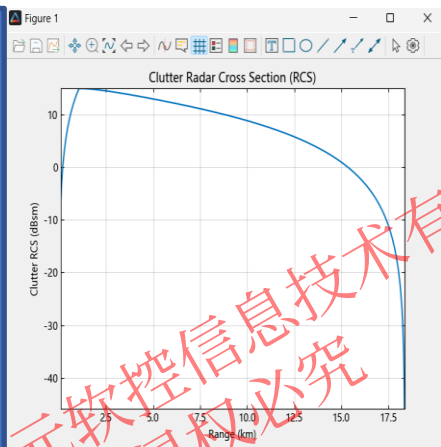


使用seareflectivity研究各种海况的反射率与频率的。将掠角设为0.5度，率设为0.5至35度。

- 1.海面反射率与正比
- 2.随着海况数的增加，粗糙度的增加，反射率也增加。

使用 clutterSurfaceRCS函数计算杂波的雷达散射截面（RCS）。

注意当达到雷达地平线范围时，杂波RCS会下降。



考虑一个工作6 GHz（C波段）平极化海上监视

模拟海况为2环境。计算并定义几何的掠入海面反射率。

部分代码

```
# Radar parameters
freq = 6e9;           # C-band frequency (Hz)
anht = 20;            # Height (m)
ppow = 200e3;         # Peak power (W)
tau = 200e-6;         # Pulse width (sec)
prf = 300;            # PRF (Hz)
azbw = 10;            # Half-power azimuth beamwidth (deg)
elbw = 30;            # Half-power elevation beamwidth (deg)
Gt = 22;              # Transmit gain (dB)
Gr = 10;              # Receive gain (dB)
nf = 3;               # Noise figure (dB)
Ts = system(nf);      # System temperature (K)

# Sea parameters
ss = 2;               # Sea state

# Calculate surface state
hgtsd,beta0 = searoughness(ss);

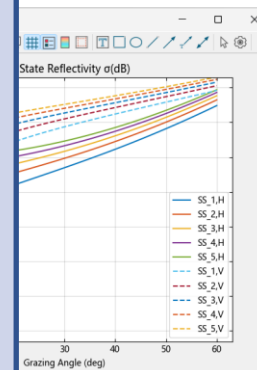
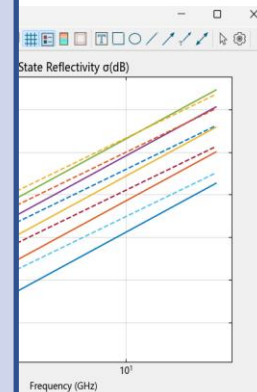
# Setup geometry
anht = anht + 2*hgtsd; # Average height above clutter (m)
surfht = 3*hgtsd;      # Surface height (m)

# Calculate maximum range for simulation
Rua = time2range(1/prf); # Maximum unambiguous range (m)
Rhoriz = horizonrange(anht, SurfaceHeight=surfht); # Horizon range (m)
Rmax = min(Rua, Rhoriz); # Maximum simulation range (m)

# Generate vector of ranges for simulation
Rm = LinRange(100, Rmax, 1000); # Range (m)
Rkm = Rm*1e-3;                  # Range (km)

# Calculate sea clutter reflectivity. Temporarily permit values outside of
# the NRL sea reflectivity model grazing angle bounds of 0.1 - 60 degrees.
# Specifically, this is to permit analysis of grazing angles less than 0.1
# degrees that are close to the horizon.
grazAng = grazingang(anht, Rm, TargetHeight=surfht);
nracs = seareflectivity(ss, grazAng, freq);
helperPlotSeaReflectivity(ss, grazAng, freq, nracs, "H");

# Calculate clutter RCS
racs = clutterSurfaceRCS(nracs, Rm, azbw, elbw, grazAng, tau);
racsdb = pow2db(racs); # Convert to decibels for plotting
ax = helperPlot(Rkm, racsdb, "RCS", "Clutter RCS (dBsm)", "Clutter Radar Cross Section (RCS)");
```



建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听